EEET2490 – Embedded System: OS and Interfacing, Semester 2025-2

Group Assignment Report

CLI, SCREEN DISPLAY, AND APPLICATION DEVELOPMENT FOR A BARE METAL OPERATING SYSTEM

Lecturer: Dr Linh Tran – linh.tranduc@rmit.edu.vn,

Team Number: 1

Team Members:

Truong Bien Hai Trong (s3872952)

Nguyen Phan Duc Anh (s3915181)

Dinh Duc Dong Nam (s3914547)

Tran Minh Triet (s3979253)

Nguyen Dinh Lam (s3990403)

Date : 10/9/2025

Table of Contents

I. Github Link **1**
II. INTRODUCTION 1
III. WELCOME MESSAGE AND COMMAND LINE INTERPRETER (CLI) 1
1. Overview of CLI: 1
2. Implemented CLI Commands 2
3. System Architecture 2
3.1 Hardware / Platform 2
3.2 Memory-Mapped I/O (key bases) 2
3.3 Boot Flow (control path) 3
3.4 Software Modules (by responsibility) 3
3.5 CLI Data & Control Design 3
3.6 Build & Run (developer workflow) 3
3.7 Assumptions & Limitations 3
4. CLI Enhancement Detail 4
4.1 Welcome Screen 4
4.2 Command Recall and Navigation 6
4.3 Auto Command Completion 9
5. Code Implementation Detail 10
5.1 Help Command (help or help <command-name>) 10
5.2 Clear Screen Command (clear) 14
5.3 Board Information Command (showinfo) 14
5.4 Baud Rate Configuration Command (baudrate <rate>) 17
5.5 Hardware Flow Control Command (handshake <on|off>) 18
IV. IMAGE, VIDEO, AND TEXT DISPLAY 20
Summary of features implemented in both Tasks 1 & 2 22
V. APPLICATION DEVELOPMENT 24
1. Gameplay Overview 24
2. Game Design and Implementation 24
3. Result Discussion 25
VI. CONCLUSION 27
VII. REFERENCES (USE IEEE STYLES) 28

I. Github Link

https://github.com/billybean93/Bare-Metal-Programming.git

II. INTRODUCTION

As part of a course on embedded systems, this report details the design and construction of a modest, educational bare-metal operating system for the Raspberry Pi 3/4 platform. We configured memory-mapped peripherals and wrote a basic kernel that boots from kernel8.img in order to directly program against the ARMv8-A hardware without the use of Linux or any other third-party runtime. A mailbox-configured framebuffer with custom drawing primitives and bitmap rendering, (1) bring-up and a UART-driven command-line interface (CLI), and (2) an interactive 2D game that tests timing, input, graphics, and fundamental physics on actual hardware comprise the project's three progressive tasks that develop core competencies. We employed double-buffered rendering to lessen flicker and tearing, used the mailbox property interface for board inquiries and graphics setup, and created low-level device drivers (PL011 UART) for all of these activities.

The project's focus is on comprehending the distinction between hardware and software. We demonstrated dependable serial I/O and fundamental shell ergonomics in Task 1 by implementing a PL011 UART driver (8-N-1, adjustable baud rates) and a lightweight CLI with commands like showinfo (board revision / MAC via mailbox), baudrate, and handshake. We created routines to draw pixels, rectangles, circles, text, and bitmaps in Task 2, which introduces a 32-bit RGBA framebuffer controlled by mailbox tags. We then animated sprite frames to verify throughput and synchronization. These subsystems are combined in Task 3 to create a small Flappy Bird-style game: a main loop powered by the architectural counters (cntpct_el0/cntfrq_el0) maintains smooth visuals, non-blocking UART input gives control, collision detection determines the gameplay state, and double buffering advances physics and animation.

III. WELCOME MESSAGE AND COMMAND LINE INTERPRETER (CLI)

We designed and implemented a functional Command Line Interpreter (CLI) tailored for a bare-metal operating system. Serving as a core element of modern operating systems, the CLI establishes a direct, text-based interface that enables users to interact seamlessly with underlying hardware and system services. In this project, our CLI was extended beyond basic input handling to include essential commands such as **help**, **clear**, **showinfo**, **baudrate**, and **handshake**, along with enhancements like auto-completion, command history navigation, and a persistent operating system prompt. These features collectively provided a user-friendly and robust foundation for system control and further development.

1. Overview of CLI:

- **Welcome Screen**:
  - Displays a custom ASCII-art banner on boot (include a screenshot).
  - Shows team credits beneath the banner (“Developed by Group 07 – COSC Embedded Systems”).
  - Boot flow: ROM/GPU → load kernel8.img → UART init → print banner → hint (“Type help to see commands.”).

- **Command Prompt**:
  - Fixed prompt string: MyOS> (hard-coded), shown after every command completes.

- I/O conventions: uses \r\n for newlines; echoes typed characters; normalizes \r → \n for consistent Enter behavior across terminals.

- **Input Handling**:
  - Line buffer size: **MAX_LINE = 128** characters.
  - Argument tokenizer: up to **MAX_ARGS = 8** whitespace-separated tokens.
  - Command history queue: **HIST_SIZE = 8** recent lines.
  - Editing & navigation: **Backspace** supported; **Tab-completion** (auto-complete or list candidates); **History** (press _ for previous, + for next).
  - Entering 'help [command]' will show the details of that command.
  - Processing flow: **keystroke → line buffer → tokenizer → dispatcher** (deterministic, bounded buffers)

- **Command Execution**:
  - Press **Enter** to submit → line is parsed (command + args) → matched in registry → handler runs.
  - Unknown commands print an error and suggest help.
  - After any command finishes, the prompt MyOS> is re-printed and the CLI returns to idle read state.

2. Implemented CLI Commands

Currently, the CLI has the placeholders `task2` and `task3` in addition to the commands `help`, `clear`, `showinfo`, `baudrate`, and `handshake`. A clear screen is sent by `clear`; a board revision and MAC are printed by `showinfo` (QEMU may remove fields); a baudrate of <9600|19200|38400|57600|115200> `handshake ` toggles CTS/RTS; ` modifies the PL011 UART rate. For subsequent graphics/game demos, the `task*` instructions are safe no-ops.

Before the prompt comes back, each command is tokenized, verified, sent, and reports a clear outcome. If ANSI escapes aren't supported, `clear` gently degrades; unknown names recommend `help`; `baudrate` only accepts whitelisted values and advises users to adapt their terminal following a change.

3. System Architecture

3.1 Hardware / Platform

Task 1 targets a Raspberry Pi 3–class board running in AArch64 bare-metal mode (no OS, no libc). The system communicates exclusively through the PL011 UART0 for console I/O and uses the Mailbox property interface to query board information. A serial terminal (typically 115200-8-N-1) provides the user interface for boot messages and the CLI. Development and testing are performed on QEMU with the raspi3b machine model, with the same binary usable on real hardware.

3.2 Memory-Mapped I/O (key bases)

Peripherals are accessed via fixed MMIO addresses. On Raspberry Pi 3 the peripheral base is 0x3F000000; the PL011 UART is mapped at UART0_BASE = 0x3F201000, and the Mailbox controller at MBOX_BASE = 0x3F00B880. The UART driver configures line control (8-N-1), baud

divisors, and FIFO behavior directly via these registers, while the Mailbox code constructs tag messages to retrieve properties such as the board revision and MAC address.

3.3 Boot Flow (control path)

At power-on the GPU bootloader loads kernel8.img, transferring control to _start in boot.S. Startup code selects core 0, sets up the stack, clears the BSS segment, and branches to main(). The kernel then initializes UART, prints an ASCII welcome banner with a brief hint (Type 'help' …), and hands control to the CLI loop. From this point, the system cycles deterministically through **Prompt → Input → Parse → Dispatch → Output → Prompt**.

3.4 Software Modules (by responsibility)

The codebase is organized to keep hardware access and shell logic separate. The **kernel** module (startup, linker script, basic utilities) coordinates initialization and user hand-off. The **UART** module provides character/line I/O, baud-rate reconfiguration, CRLF normalization, and optional CTS/RTS toggling. The **Mailbox** module implements the property channel used by showinfo. The **CLI** module implements the prompt, line editing, tokenization, a command registry (name → handler), and formatted responses.

3.5 CLI Data & Control Design

The CLI exposes a fixed prompt string (MyOS>) and processes input through bounded buffers for predictability. A line buffer of **MAX_LINE = 128** characters is tokenized into at most **MAX_ARGS = 8** arguments; a small history (**HIST_SIZE = 8**) aids interactive use. Basic editing (echo, Backspace), history navigation, and tab-completion are supported. Implemented handlers cover help, clear, showinfo (Mailbox), baudrate (whitelisted rates), and handshake (CTS/RTS), with task2/task3 reserved as safe placeholders.

3.6 Build & Run (developer workflow)

The project is built with an AArch64 bare-metal toolchain (aarch64-none-elf-gcc) using freestanding flags (-ffreestanding -nostdlib) and a custom link script to produce kernel8.img. For emulation, QEMU runs the image with qemu-system-aarch64 -M raspi3b -kernel kernel8.img -serial stdio, mapping the UART to the host terminal. The same artifact can be deployed to a real board via the standard Raspberry Pi boot partition.

3.7 Assumptions & Limitations

Task 1 uses polling rather than interrupts or DMA, prioritizing simplicity and control-flow clarity. Buffer sizes and history depth are fixed to keep memory usage bounded. ANSI escape sequences (used by clear) rely on terminal support; non-ANSI terminals degrade gracefully. Some Mailbox properties may be unavailable under QEMU but function as expected on physical hardware.

4. CLI Enhancement Detail

4.1 Welcome Screen

At system initialization, the terminal presents a custom ASCII banner that acts as a welcome screen. The ASCII graphics were generated using an online conversion tool provided in the assignment resources, then embedded as a character array within the code. This array is printed automatically during boot, giving the system a distinctive visual identity.

```
static void print_welcome(void)
{
    uart_puts("\n");
    uart_puts(" ######## ######## ########  ######  #        ######    ###   \n");
    uart_puts(" #       #          #    #     # #    #  #     #  #   #  \n");
    uart_puts(" #       #          #          # #    #  #     # #     # \n");
    uart_puts(" ######   ######     #     ######  #    #   ###### #     # \n");
    uart_puts(" #       #          #    #       ########       # #     # \n");
    uart_puts(" #       #          #    #             #  #     #  #   #  \n");
    uart_puts(" ######## ########    #    ########      #   ######    ###   \n");
    uart_puts("                                                           \n");
    uart_puts("                                                           \n");
    uart_puts(" #######     #    #######  ########    ########  ######       \n");
    uart_puts(" #     #   # #   #     # #          #     # #     #      \n");
    uart_puts(" #     #  #   #  #     # #          #     # #            \n");
    uart_puts(" #######  #     # #######  ######     #     #  ######       \n");
    uart_puts(" #     # ######## #   #   #          #     #       #      \n");
    uart_puts(" #     # #     # #    #  #          #     # #     #      \n");
    uart_puts(" #######  #     # #     # ########    ########  ######       \n");
    uart_puts("\n");
    uart_puts("     Developed by Group 07 - COSC Embedded Systems       \n");
    uart_puts("\n");
}
```

Figure 1: welcome screen (in code)

Figure 2: welcome screen (in VS Code terminal)

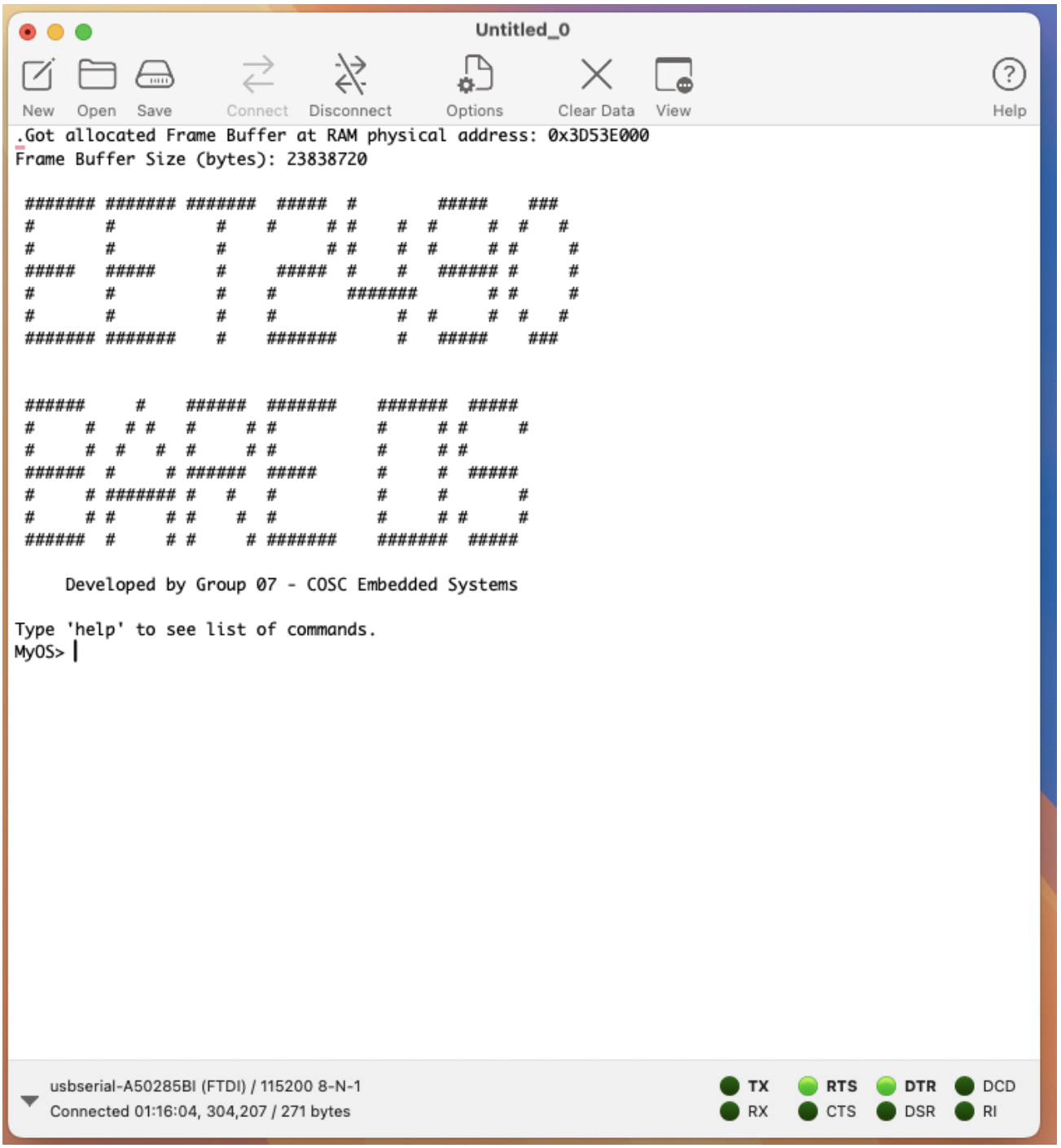

Figure 3 : Welcome screen (in TeraTerm/CoolTerm)

4.2 Command Recall and Navigation

The CLI also includes a persistent command history. Every time a command is executed, it is stored in an internal history buffer. Users can revisit earlier inputs by pressing the "+" and "_" keys, allowing them to re-execute or modify commands without retyping them. This makes the system more efficient and user-friendly, especially during testing and debugging.

```
    static char history[HIST_SIZE][MAX_LINE];
    static int hist_count = 0;
    static int hist_index = 0;
```

```
static void print_prompt(void) { uart_puts(PROMPT); }

static void put_hex32(uint32_t v)
{
    const char *hex = "0123456789ABCDEF";
    for (int i = 28; i >= 0; i -= 4)
    {
        uart_sendc(hex[(v >> i) & 0xF]);
    }
}

static void put_mac(uint8_t mac[6])
{
    const char *hex = "0123456789ABCDEF";
    for (int i = 0; i < 6; i++)
    {
        uart_sendc(hex[(mac[i] >> 4) & 0xF]);
        uart_sendc(hex[mac[i] & 0xF]);
        if (i < 5)
            uart_sendc(':');
    }
}
static command_t *find_command(const char *name)
{
    for (int i = 0; i < NCMDS; i++)
        if (u_strcmp(name, commands[i].name) == 0)
            return &commands[i];
    return 0;
}

static int tokenize(char *line, char **argv)
{
    int argc = 0;
    char *p = line;
    while (*p && argc < MAX_ARGS)
    {
        while (*p && u_isspace(*p))
            *p++ = 0;
        if (!*p)
            break;
        argv[argc++] = p;
        while (*p && !u_isspace(*p))
            p++;
    }
    return argc;
}

static void add_history(const char *line)
{
    if (!line || !*line)
        return;
    int slot = hist_count % HIST_SIZE;
    int i = 0;
    while (line[i] && i < MAX_LINE - 1)
    {
        history[slot][i] = line[i];
        i++;
```

```
        }
        history[slot][i] = 0;
        hist_count++;
        hist_index = hist_count;
    }

    static int history_get_prev(char *buf)
    {
        if (hist_count == 0)
            return 0;
        if (hist_index == 0)
            hist_index = 0;
        else
            hist_index--;
        int idx = (hist_index % HIST_SIZE + HIST_SIZE) % HIST_SIZE;
        int i = 0;
        while (history[idx][i])
        {
            buf[i] = history[idx][i];
            i++;
            if (i >= MAX_LINE - 1)
                break;
        }
        buf[i] = 0;
        return 1;
    }
    static int history_get_next(char *buf)
    {
        if (hist_count == 0)
            return 0;
        if (hist_index < hist_count)
            hist_index++;
        if (hist_index == hist_count)
        {
            buf[0] = 0;
            return 1;
        }
        int idx = (hist_index % HIST_SIZE + HIST_SIZE) % HIST_SIZE;
        int i = 0;
        while (history[idx][i])
        {
            buf[i] = history[idx][i];
            i++;
            if (i >= MAX_LINE - 1)
                break;
        }
        buf[i] = 0;
        return 1;
    }
    static void redraw_line(const char *buf)
    {
        uart_puts("\r\n");
        print_prompt();
        uart_puts(buf);
    }
```

Figure 4: Command History Navigation code block

4.3 Auto Command Completion

To simplify user interaction, the CLI supports predictive completion. Whenever the user presses the **TAB** key, the current input string is compared against all registered commands. If a valid match is detected, the interpreter fills in the remaining characters automatically. This feature reduces typing effort and minimizes command errors.

```
/* Tab completion: if one match, complete; if many, list */
static void complete(char *buf, int *len)
{
    int matches[16];
    int m = 0;
    for (int i = 0; i < NCMDS; i++)
    {
        if (u_strncmp(commands[i].name, buf, (size_t)*len) == 0)
        {
            if (m < 16)
                matches[m++] = i;
        }
    }
    if (m == 0)
        return;
    if (m == 1)
    {
        /* complete */
        const char *name = commands[matches[0]].name;
        int n = (int)u_strlen(name);
        for (int i = *len; i < n && i < MAX_LINE - 1; i++)
        {
            buf[i] = name[i];
        }
        for (int i = *len; i < n; i++)
        {
            uart_sendc(name[i]);
        }
        *len = n;
        buf[*len] = 0;
    }
    else
    {
        uart_puts("\r\n");
        for (int i = 0; i < m; i++)
        {
            uart_puts(commands[matches[i]].name);
            uart_puts("  ");
        }
        redraw_line(buf);
    }
}
```

Figure 5: Autocomplete code block

5. Code Implementation Detail

5.1 Help Command (help or help <command-name>)

The **help** command provides users with guidance on available system commands. When executed without parameters, it generates a complete list of supported commands along with a brief description of their functionality. Alternatively, if the command is followed by a specific keyword (e.g., **help showinfo**), the CLI returns detailed usage instructions and an explanation tailored to that particular command. This dual functionality ensures both new and experienced users can easily understand and operate the system.

```
static int cmd_help(int argc, char **argv)
{
    if (argc == 2)
    {
        command_t *c = find_command(argv[1]);
        if (!c)
        {
            uart_puts("Unknown command\n");
            return 0;
        }
        uart_puts(c->help);
        uart_puts("\n");
        return 0;
    }
    uart_puts("Commands:\n");
    for (int i = 0; i < NCMDS; i++)
    {
        uart_puts("  ");
        uart_puts(commands[i].help);
        uart_puts("\n");
    }
    return 0;
}
```

Figure 6: Help command's code

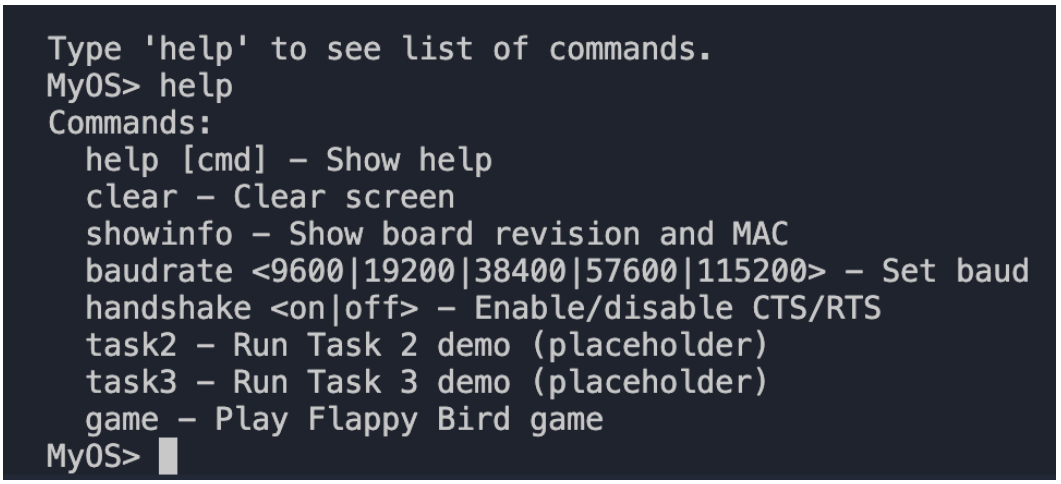

Figure 7: help command function (in VS Code terminal)

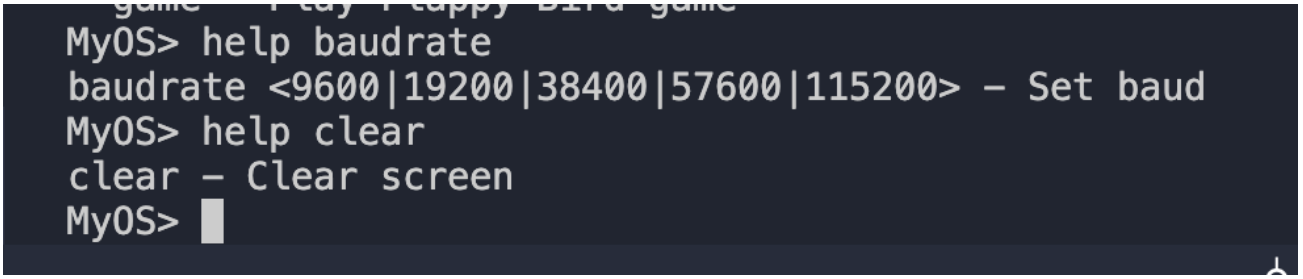

Figure 8: help [clear] command function (in VS Code terminal)

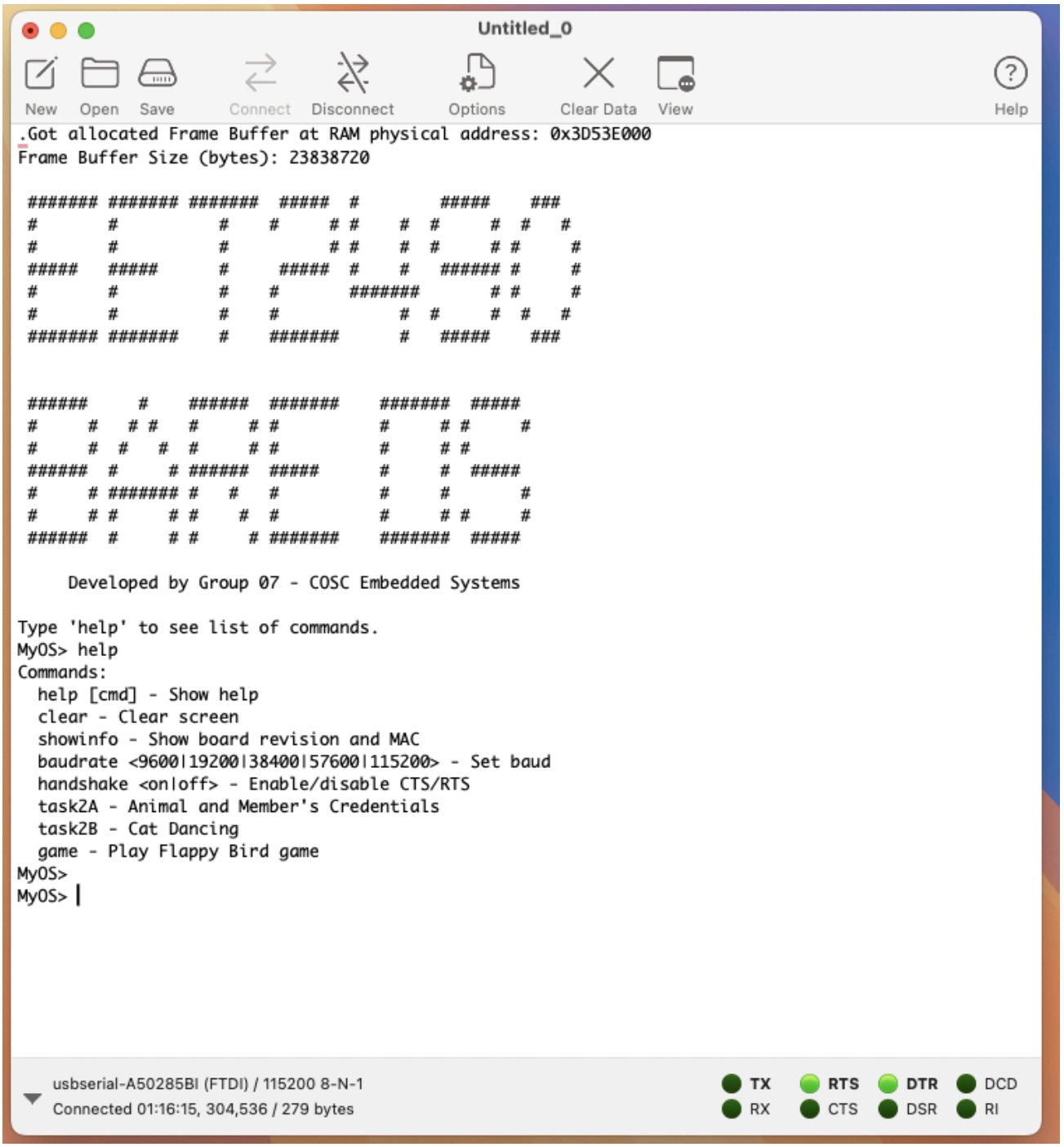

Figure 9: help command function (in TeraTerm/CoolTerm)

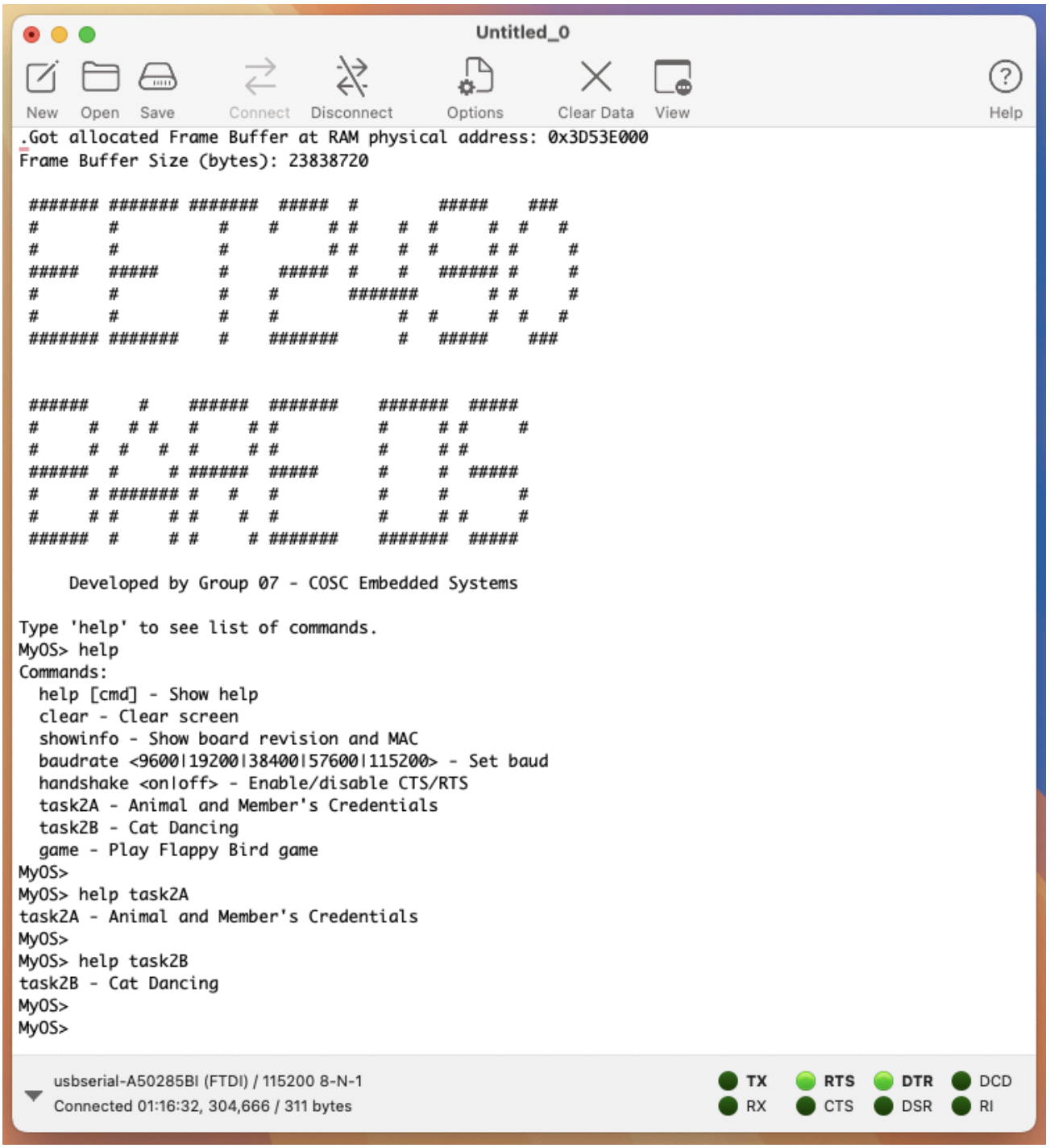

Figure 10: help [task2A & task2B] command function (in TeraTerm/CoolTerm)

5.2 Clear Screen Command (clear)

The **clear** command refreshes the terminal display by pushing a series of blank lines onto the screen, effectively removing previously visible content. This provides the user with a clean working area and improves readability during extended system interaction.

```
static int cmd_clear(int argc, char **argv)
{
    /* ANSI clear screen + home; if terminal doesn't support, it's harmless */
    uart_puts("\033[2J\033[H");
    return 0;
}
```

Figure 11: clear command code

5.3 Board Information Command (showinfo)

The **showinfo** command was implemented to retrieve and present key hardware identifiers from the Raspberry Pi board, namely the board revision and the MAC address. This functionality relies on mailbox communication to query the firmware and is organized into modular helper functions:

- **Board Revision Retrieval**
  The function **mbox_get_board_revision()** constructs a mailbox message with the tag **TAG_GET_BOARD_REVISION** (**0x00010002**). Once the request is processed, the revision code is returned in the mailbox buffer. This value is then used to identify the specific board version.
- **MAC Address Retrieval**
  The function **mbox_get_mac()** issues a mailbox request with the tag **TAG_GET_MAC_ADDRESS** (**0x00010003**). The response splits the 6-byte MAC address across two fields: the high four bytes (**mac_hi**) and the lower two bytes (**mac_lo**). These values are combined to reconstruct the complete MAC address. If the query fails, the system falls back to a predefined default address.
- **Output Formatting**
  After successful retrieval, the revision and MAC address are passed to dedicated printing routines that format the results into a human-readable string. The information is then displayed on the terminal, ensuring the user has clear insight into the hardware configuration of the system.

This design leverages low-level firmware communication while keeping the command modular and reliable, making **showinfo** a practical diagnostic feature for the CLI.

```
int mbox_get_board_revision(uint32_t *rev)
{
    mbox[0] = 8 * 4;
    mbox[1] = 0x00000000; /* request */
    mbox[2] = TAG_GET_BOARD_REVISION;
    mbox[3] = 4;
    mbox[4] = 0;
    mbox[5] = 0;
```

```
    mbox[6] = TAG_END;

    if (mbox_call(8))
    {
        *rev = mbox[5];
        return 0;
    }
    return -1;
}

int mbox_get_mac(uint8_t mac[6])
{
    mbox[0] = 9 * 4;
    mbox[1] = 0x00000000;
    mbox[2] = TAG_GET_MAC_ADDRESS;
    mbox[3] = 6;
    mbox[4] = 0;
    mbox[5] = 0; /* bytes 0..3 */
    mbox[6] = 0; /* bytes 4..5 in low halfword */
    mbox[7] = TAG_END;

    if (mbox_call(8))
    {
        unsigned int hi = mbox[5];
        unsigned int lo = mbox[6];
        mac[0] = (hi >> 24) & 0xFF;
        mac[1] = (hi >> 16) & 0xFF;
        mac[2] = (hi >> 8) & 0xFF;
        mac[3] = (hi >> 0) & 0xFF;
        mac[4] = (lo >> 8) & 0xFF;
        mac[5] = (lo >> 0) & 0xFF;
        return 0;
    }
    return -1;
}
```

Figure 12: get_board_revision() and get_mac_address() function

```
static int cmd_showinfo(int argc, char **argv)
{
    uint32_t rev = 0;
    uint8_t mac[6] = {0};
    int ok1 = mbox_get_board_revision(&rev);
```

```
    int ok2 = mbox_get_mac(mac);
    uart_puts("Board revision: 0x");
    put_hex32(rev);
    if (ok1 != 0)
        uart_puts(" (unavailable in QEMU)");
    uart_puts("\nMAC: ");
    if (ok2 == 0)
    {
        put_mac(mac);
    }
    else
    {
        uart_puts("unavailable in QEMU");
    }
    uart_puts("\n");
    return 0;
}
```

Figure 13: show information command function

```
MyOS> showinfo
Board Revision: 0x00A02082
MAC Address: 18:00:84:82:87:52
Board revision: 0x00000000 (unavailable in QEMU)
MAC: unavailable in QEMU
MyOS>
```

Figure 14: show information command function (in VS Code terminal)

```
MyOS> showinfo
Board revision: 0x0x00A22082
MAC: A6:EB:27:B8:41:83
MyOS>
MyOS>
```

Figure 15: show information command function (in TeraTerm/CoolTerm)

5.4 Baud Rate Configuration Command (baudrate <rate>)

The **baudrate** command enables runtime adjustment of the UART communication speed, providing flexibility for different serial terminal configurations. This implementation supports commonly used baud rates, including **9600, 19200, 38400, 57600, and 115200 bps**.

Implementation Details:

- **Input Validation**: The command accepts a single argument representing the desired baud rate. If no argument or multiple arguments are given, the system responds with a usage prompt (**baudrate <9600|19200|38400|57600|115200>**).
- **Parsing**: Each character of the argument string is checked to confirm it is numeric. Non-digit characters trigger an "Invalid baud" error message.
- **Baud Rate Conversion**: Valid numeric input is converted into an integer value, which is then passed to the function **uart_set_baud()**.
- **Configuration**:
    - If the requested rate is supported, the system updates the UART registers accordingly and notifies the user with confirmation.
    - The message also instructs the user to switch their terminal emulator to the new baud rate and press Enter to resume communication.
    - If the input does not match a supported baud rate, the system prints an "Unsupported baud" warning.

This design ensures that baud rate changes are performed safely and only when valid parameters are provided, while maintaining clear feedback to the user for proper reconfiguration of the terminal connection.

```
static int cmd_baudrate(int argc, char **argv)
{
    if (argc != 2)
    {
        uart_puts("Usage: baudrate <9600|19200|38400|57600|115200>\n");
        return 0;
    }
    const char *s = argv[1];
    unsigned int b = 0;
    for (int i = 0; s[i]; i++)
    {
        if (!u_isdigit(s[i]))
        {
            uart_puts("Invalid baud\n");
            return 0;
        }
        b = b * 10 + (unsigned)(s[i] - '0');
    }
    if (uart_set_baud(b) == 0)
```

```
    {
        uart_puts("OK. Please change your terminal to ");
        uart_puts(argv[1]);
        uart_puts(" and hit Enter.\n");
    }
    else
    {
        uart_puts("Unsupported baud.\n");
    }
    return 0;
}
```

Figure 16: Set baud rate command

```
MyOS> baudrate
Usage: baudrate <9600|19200|38400|57600|115200>
MyOS> baudrate 9600
OK. Please change your terminal to 9600 and hit Enter.
```

Figure 17: Set baud rate command (in VS Code terminal)

```
MyOS> baudrate
Usage: baudrate <9600|19200|38400|57600|115200>
MyOS>
MyOS> baudrate 9600
OK. Please change your terminal to 9600 and hit Enter.
MyOS>
MyOS>
```

Figure 18: Set baud rate command (in TeraTerm/CoolTerm)

5.5 Hardware Flow Control Command (handshake <on|off>)

The **handshake** command allows users to enable or disable UART hardware flow control during runtime. This feature manages the **CTS (Clear To Send)** and **RTS (Request To Send)** signals, ensuring reliable communication between devices.

Implementation Details:

- **Argument Handling**: The command expects a single parameter. If the user input is missing or incorrect, the CLI responds with usage instructions (**handshake <on|off>**).
- **Enabling Handshake**: When the argument is **on**, the function **uart_set_flow(1)** is called to activate CTS/RTS flow control, followed by a confirmation message (**CTS/RTS enabled**).

- **Disabling Handshake**: When the argument is **off**, the function **uart_set_flow(0)** disables the flow control mechanism, and the terminal outputs (**CTS/RTS disabled**).
- **Error Handling**: Any input that does not match the expected values (**on** or **off**) results in the usage message being reprinted, guiding the user back to the correct format.

This design ensures that hardware flow control can be quickly configured without requiring system restarts or low-level register manipulation by the user, improving usability and flexibility in testing environments.

```
static int cmd_handshake(int argc, char **argv)
{
    if (argc != 2)
    {
        uart_puts("Usage: handshake <on|off>\n");
        return 0;
    }
    if (u_strcmp(argv[1], "on") == 0)
    {
        uart_set_flow(1);
        uart_puts("CTS/RTS enabled\n");
    }
    else if (u_strcmp(argv[1], "off") == 0)
    {
        uart_set_flow(0);
        uart_puts("CTS/RTS disabled\n");
    }
    else
        uart_puts("Usage: handshake <on|off>\n");
    return 0;
}
```

Figure 19: Handshake command function

```
MyOS> help handshake
handshake <on|off> - Enable/disable CTS/RTS
MyOS> handshake on
CTS/RTS enabled
MyOS> handshake off
CTS/RTS disabled
MyOS>
```

Figure 20: Handshake command function (in VS Code terminal)

```
MyOS> handshake on
CTS/RTS enabled
MyOS>
MyOS> handshake off
CTS/RTS disabled
MyOS>
MyOS>
```

Figure 21: Handshake command function (in TeraTerm/CoolTerm)

IV. IMAGE, VIDEO, AND TEXT DISPLAY

In this section, we have made a screen for image and text displaying our selected animals and our names at the bottom of each selected animal. Additionally, a video of a cat dancing is also implemented. Below are the running commands for task 2 and task 3(2nd task of Task 2), for clarity we separated the image and text into a different task and video into a separate task.

Figure 22: Task2 Image and Text Display (in QEMU)

The code for Task2 Image and Text display is featured below, first we initiate the drawbitmap which displays the image of the animals. The total width of the image is 1024 with the height is 669, which leaves the remaining height level for the names and the sIDs.

```
void drawBgAndNames(){
    drawBitmap(0, 0, BACKGROUND_IMAGE_WIDTH, 669,  background_img);
    drawString(15, 680, "ANH NGUYEN", 0xFFFFFF, 2);
    drawString(15, 700, "S3915181", 0xFFFFFF, 2);

    drawString(200, 680, "TRONG TRUONG", 0x747eaf, 2);
    drawString(200, 700, "S3872952", 0x747eaf, 2);

    drawString(426, 680, "TRIET TRAN", 0xefa3a3, 2);
    drawString(426, 700, "S3979253", 0xefa3a3, 2);

    drawString(635, 680, "LAM NGUYEN", 0xe6d2a8, 2);
    drawString(635, 700, "S3990403", 0xe6d2a8, 2);

    drawString(855, 680, "NAM DINH", 0xa8e6ce, 2);
    drawString(855, 700, "S3914547", 0xa8e6ce, 2);

}
```

Figure 23: Task2 drawBgAndNames section

We have completed the requirements of making each member's name and sID different colors from each other. Additionally, the drawBitmap() function was used to help initiate the 'background_img'. The function help loops through the pixel data of the animal bitmap array and copies each pixel into the framebuffer.

Figure 24: Task3 Video Display

The figure above displays the cat dancing, the program loops through the 10 serialized images and loops them. With the **display_video()** function we added the 'X' or 'x' when the user wants the cat to stop dancing. Since for the CLI if the cat video would not stop, then the CLI will not return

another new line in the terminal, therefore, adding the 'X' as a way to stop the task is the solution to this issue. This section of code is displayed below:

```
void display_video(int x, int y, int width, int height, const uint32_t **frames, int num_frames, int delay_count) {
    uart_puts("[Press 'x' or 'X' to stop video]\n");

    while (1) {
        for (int i = 0; i < num_frames; i++) {
            drawBitmap(x, y, width, height, frames[i]);
            delay(delay_count);

            if (uart_has_char()) {
                char c = uart_getc_nowait();
                if (c == 'x' || c == 'X') {
                    uart_puts("\nVideo stopped by user.\n");
                    return;
                }
            }
        }
    }
}
```

Figure 25: Task3 display_video() function

The figure above shows the order of the display_video() function, if initiated it will loop the number of frames and display the image through the drawBitmap. During looping it will delay the operation to make the video smoother. Additionally, below is the code section which if called in the terminal will execute the task3, there we can see the operation of the display_video() and the frame buffer. Currently, it's being set at 10 frames per second which when run would make the video of the cat dancing look smooth.

```
static int cmd_task3(int argc, char **argv)
{
    uart_puts("Task 3 launch the cat dancing video\n");
    adjustVideoScreen();
    display_video(0, 0, CAT_DANCE_WIDTH, CAT_DANCE_HEIGHT, frames_order, NUM_FRAMES, 100000000);

    return 0;
}
```

Figure 26: cmd_Task3() function

Summary of features implemented in both Tasks 1 & 2

Feature Group	Command/ Feature	Implementation	Testing (any issues/limitations)
CLI Basic Features	Welcome Screen	Implemented in Task 1/kernel/kernel.c: print_welcome() prints ASCII banner over UART right after uart_init(), followed by "Type 'help' …". CLI prompt is MyOS>.	**Test:** power on board, open serial @ **115200 8N1**, banner + hint should appear. **Limitations:** depends on terminal supporting CR/LF; no color.

	help	In Task 1/CLI/cli.c: commands[] includes "help". cmd_help prints all available commands or detailed info for one command.	Test: run help and help clear. Limitations: case-sensitive; only lists commands in the table.
	clear	"clear" invokes cmd_clear, which prints ANSI escape sequence (clear screen + reset cursor).	Test: type clear → terminal screen wiped. Limitations: requires ANSI-capable terminal (picocom/minicom/screen). Some Windows terminals need config.
	showinfo	"showinfo" calls cmd_showinfo: uses Mailbox tags GET_BOARD_REVISION and GET_MAC_ADDRESS to retrieve board revision + MAC and print over UART.	Test: run showinfo. Limitations: valid only on real Raspberry Pi; QEMU may return zeros/invalid values. Depends on GPU mailbox working.
	baudrate	baudrate < 9600	19200
	handshake	handshake < on	off>"→uart_set_flow()` enables/disables CTSEN/RTSEN (UART0_CR).
CLI Enhancement	OS name in CLI	Prompt defined by #define PROMPT "MyOS> ". Printed after each command loop.	Test: confirm prompt shows MyOS>. Limitations: name hard-coded; requires recompilation to change.
	Auto-completion in CLI	complete() checks prefix match against commands[]. One match → auto-complete; multiple matches → list them.	Test: type partial command + Tab. Limitations: prefix-only matching; max ~16 suggestions; case-sensitive.
	Command history in CLI	Input loop maintains history buffer HIST_SIZE=8, allows recall with Up/Down keys.	Test: run several commands, use ↑/↓ to navigate history. Limitations: only 8 entries, not persistent, depending on terminal escape sequences.

Image, Video, and Text Display	Background image and text display	Implemented in Task 2/kernel/framebf.c: framebf_init() sets framebuffer (RGBA32); drawBitmap() renders background_img (1024×669); drawString() overlays student names/IDs in white. Helper: drawBgAndNames().	No issue and limitation
	Video display	Video frames prepared in Task 2/bitmap/ (cat_dance1..10, frames_order[]). Frames can be drawn with drawBitmap() in a loop with delays (manual playback).	Each video frame is distorted due to the board's hardware limitations.

V. APPLICATION DEVELOPMENT

1. Gameplay Overview

A bare-metal Raspberry Pi system is used to create the game created in Task 3, which is a condensed version of Flappy Bird. A bird sprite that the player controls must flap upward to avoid running into pipes that are depicted on the screen as it falls steadily due to gravity. The goal is to squeeze through as many pipe openings as you can without running into anything or the ground.

The framebuffer transitions to graphics mode after the game is launched from the CLI. By sending a "flap" input command over UART, the player can communicate with the bird. The bird rises with each flap and falls when there is no input. This design maintains the game's difficulty while emulating the original Flappy Bird's fast-paced gameplay.

2. Game Design and Implementation

a. Management of Background

A static image created once with the framebuffer driver created in Task 2 serves as the background. It has a ground strip at the bottom of the screen and a blue sky background. Even though the background is static, the ground section's use of a repeating tile bitmap helps to create the illusion of continuous scrolling.

b. Handling Sprite

The bird is implemented as a small sprite with multiple frames (flap up, flap mid, flap down). During gameplay, the animation loop moves through these frames, creating the illusion of wing flapping. Only the bird pixels overwrite the background because the alpha channel respects transparency.

Pipe sprites drawn at predetermined intervals serve as obstacles. Every pipe is shown as a top-and-bottom vertical bitmap with a space between them. To give the appearance of forward

motion, pipes are positioned horizontally, which reduces each frame. A pipe is recycled back to the right with a fresh randomized gap position as it leaves the left side of the screen.

c. Game Logic and Interactions

Three essential components are continuously updated by the main game loop:

- Gravity: the bird descends a predetermined number of pixels with each tick.
- The flap command pushes the bird upward by setting its vertical velocity to a negative value when the player enters it in the CLI.
- Collision detection: the ground and pipe bitmaps are compared with the bird's bounding box. The game terminates and a "Game Over" message is displayed if overlap happens.

Every time the bird successfully navigates a pipe gap, the score is increased. The drawString() function is used to display the score as a text overlay at the top of the screen.

3. Result Discussion

a. Implementation Results

The Flappy Bird game effectively illustrates how a fully interactive application can be supported by the bare-metal environment created in Tasks 1 and 2. The framebuffer driver made it possible to draw backgrounds, sprites, and score overlays smoothly. Basic but useful player input was possible with the CLI. The game captures the fundamental "feel" of Flappy Bird, including gravity, flapping, pipes, and scoring, even without sophisticated libraries.

b. Testing Results

- The graphics on QEMU worked, but because of variations in the delay loops, the sprite timing was erratic.
- The pipe scrolling and bird's animation functioned as planned on the Raspberry Pi hardware. Although UART typing added a noticeable latency when compared to conventional button input, the gameplay was responsive.
- When you hit a pipe or the ground, the game ended instantly and the Game Over routine was activated. This demonstrated how reliable the collision detection was.

C. Limitations

- Input method: pressing a button causes faster reaction times than typing a flap in the CLI.
- Performance: Without hardware timers or double buffering, animations may occasionally look choppy because frame rate is limited by software delay loops.
- Graphics: there is no smooth ground animation or parallax scrolling; the pipes and background are straightforward bitmap drawings.
- Lack of persistence: no high score tracking is used, and the score is reset after every run.

Even with these constraints, the bare-metal Flappy Bird project demonstrates a complete application pipeline: from input parsing and game logic to real-time graphics rendering. It validates the successful integration of CLI, framebuffer graphics, and sprite handling into a cohesive interactive game, aligning with the learning outcomes of the assignment.

Figure 27: Flappy Bird Screen

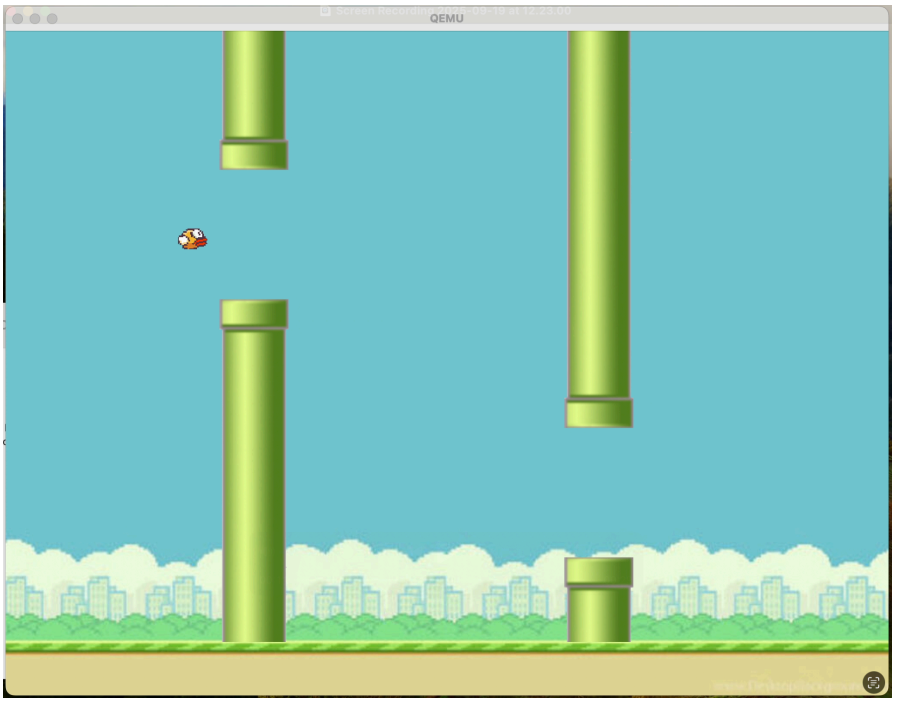

Figure 28: Flappy Bird Screen with Pipe

Figure 29: Flappy Bird Game Over Screen

VI. CONCLUSION

In conclusion, this project has allowed us to bridge theory with practice by designing and implementing a bare-metal operating system environment on the Raspberry Pi board. Starting with the Command Line Interpreter (CLI), we developed a responsive interface that supports essential commands, auto-completion, history navigation, and ensuring an initiative foundation for system control. We then advanced to graphics programming, learning how to manipulate the framebuffer directly to render images, text, and video/animation while applying double buffering techniques to reduce flickering. For our game, we integrated these components into a functional "Flappy Bird" game, which tested our ability to combine sprite animation, real-time input handling, collision detection, and scoring into a cohesive interactive application.

This project provided us with practical exposure to essential low-level concepts, including memory-mapped I/O, UART communication, mailbox property channels, and synchronization using architectural timers. Working across both QEMU and real Raspberry Pi hardware emphasized the differences between simulated and physical environments, requiring us to adapt our implementations and refine our solutions under real-world constraints. In addition to technical development, we also enhanced our teamwork, version control practices, and problem-solving skills through continuous collaboration.

Through this assignment and the course as a whole, we developed a deeper understanding of embedded operating systems and hardware–software integration. These competencies are directly applicable to global career opportunities in embedded software engineering, where professionals are expected to design efficient, reliable, and low-level solutions for domains such as IoT, automotive systems, and robotics. Looking ahead, we are motivated to expand our knowledge in interrupt-driven architectures, real-time scheduling, and hardware acceleration, as these areas will further strengthen both our academic foundation and career readiness. Ultimately, this project not only advanced our

technical expertise but also inspired us to continue exploring the broader applications and future possibilities of embedded systems.

VII. REFERENCES (USE IEEE STYLES)